CS-107 Lecture 3
Number Systems

Troy Astarte

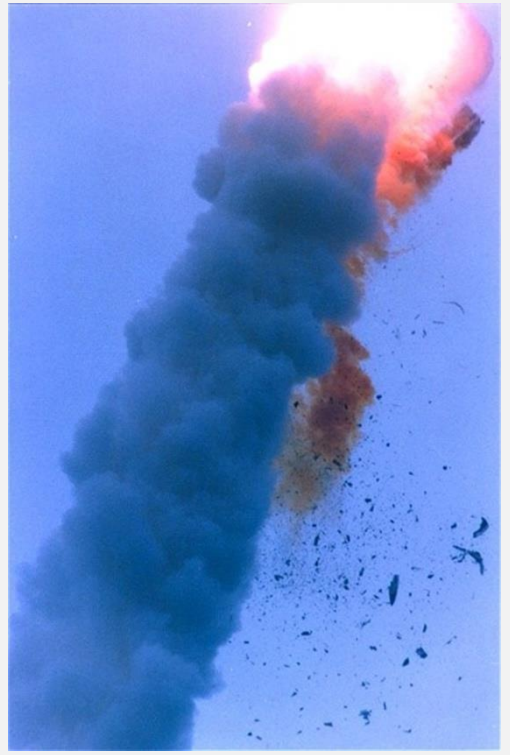
Megan Venn-Wycherley

What is data representation?

- Data representation is the process of presenting information in a meaningful way, allowing for easy understanding and analysis.
- It involves organising and structuring data in a format that can be interpreted by computers or humans.
- By selecting the right representation methods and following best practices, we can communicate and understand data more efficiently.

Why care about data representation?

- **4 June 1996:** Ariane 5 explodes 37 seconds after launch, destroying $370M of rocket and satellites.
- **Cause:** a 64-bit floating-point velocity was written into a 16-bit signed integer. The number did not fit - the same code had flown safely on Ariane 4, which never went fast enough to overflow the field.
- No hardware failure – just one conversion between representations lost the mission.



Source: ESA/CNES Ariane 501 Inquiry Board report, July 1996.

Types of data representation

1. Textual: Representing data using words, sentences, or paragraphs.
2. Numerical: Representing data through numbers or mathematical values.
3. Visual: Representing data using graphs, charts, maps, or diagrams.
4. Multimedia: Representing data through a combination of text, images, audio, and video.

Number Systems

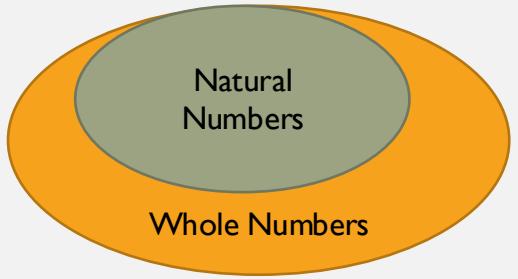
Goals for Today

- Categories of numbers and positional notation
- Overview of bases 2, 8, and 16
- Converting numbers between bases

Numbers

- **Natural Numbers ℕ:**
  - The counting numbers achieved by adding 1
  - {1, 2, 3, 4, …}

- **Whole Numbers 𝕎:**
  - The natural numbers AND zero
  - Also called non-negative integers
  - {0, 1, 2, 3, 4, …}



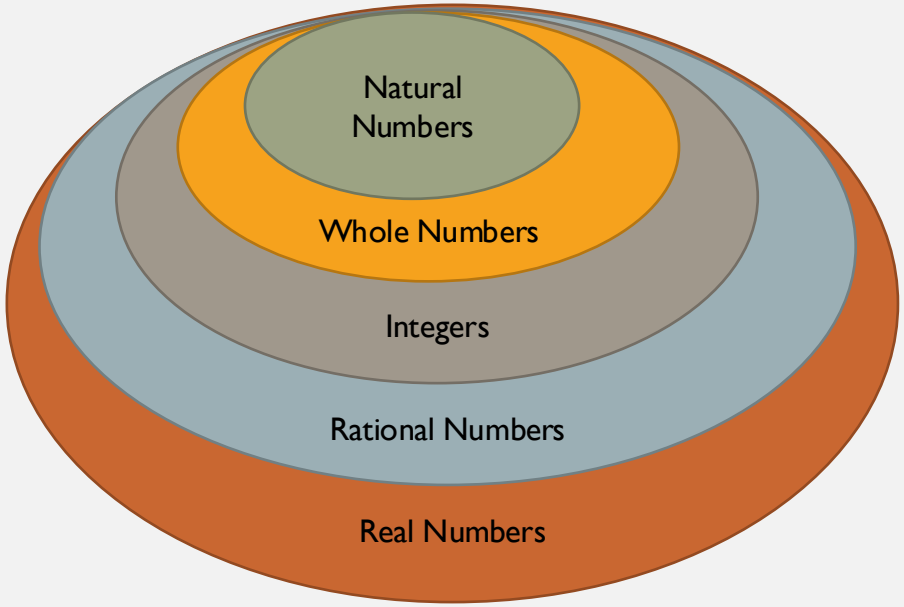
This classification can cause arguments amongst mathematicians

Numbers

- Integers $\mathbb{Z}$:
  - The Whole Numbers and negative Natural Numbers
  - {…, -4, -3, -2, -1, 0, 1, 2, 3, 4, …}

- Rational Numbers $\mathbb{Q}$:
  - Integer, or a quotient of an integer and a non-zero integer.
  - {Integers, 3/8, -5.28, …}

- Real Numbers $\mathbb{R}$:
  - All "non-imaginary" numbers



Writing numbers

- When writing a number we use **digits**: {0, 1, 2, 3, …, 9} and the number of digits available to us defines the **base** of the number we are representing:
  - Base 2: {0, 1} (binary)
  - Base 3: {0, 1, 2}
  - …
  - Base 10: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} (decimal)
  - …
  - Base 16: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F} (hexadecimal)

Writing numbers

- To avoid **ambiguity**, we can state the base a number is in.

  Is "357" in base 8? base 10? base 16?

- We write number in brackets (sometimes) and include the base in **subscript**:

$$(101100101)_2 = (357)_8 = (239)_{10} = (165)_{16}$$

Writing bigger numbers

- If we have only 10 digits, how do we write larger decimal numbers?
- In decimal, for any number **greater** than 9, we need to use multiple digits…
- We use **positions** to modify what that digit **represents**:

357

This is really 3 hundreds, 5 tens, and 7 ones…

Positional Notation

Positional Notation

- Represent a number by **digits** in **positions**, indexed from the **rightmost** position. Increase in position results in increase in magnitude:

Decimal example:

Units	10^0	=	1
Tens	10^1	=	10
Hundreds	10^2	=	100
Thousands	10^3	=	1000
…			

The superscript here denotes the position

Positional Notation

- To get our value, we multiply the digit at a position by the base raised to the power of its position:

- E.g. in **decimal** (base 10):

$$357 = 3 \cdot 10^2 + 5 \cdot 10^1 + 7 \cdot 10^0$$

- General form, in base 10, an *n* digit number is represented as:

$$d_{n-1} \ldots d_2 d_1 d_0 = d_{n-1} \cdot 10^{n-1} + \ldots + d_2 \cdot 10^2 + d_1 \cdot 10^1 + d_0 \cdot 10^0$$

Generalisation to other bases

- A base is a representation scheme that describes the available number of unique symbols available to represent numbers.

- In **decimal** (base 10) we have 10 unique symbols: {0,1,2,…,9}
- In **octal** (base 8) we have 8 unique symbols: {0,1,2,…,7}

- The general form of positional notation for base *b*:

$$d_{n-1}\ldots d_2 d_1 d_0 = d_{n-1} \cdot b^{n-1} + \ldots + d_2 \cdot b^2 + d_1 \cdot b^1 + d_0 \cdot b^0$$

$$\text{(base b = 10)}\ 1_2 5_1 6_0 = 1_2 \cdot 10^2 + 5_1 \cdot 10^1 + 6_0 \cdot 10^0$$

Number Bases

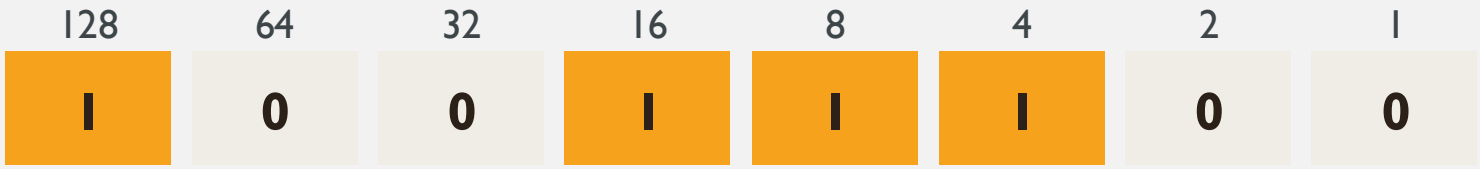
Useful to Computer Science

There's a number of bases that are useful to computer science. Any ideas what these could be?



Binary

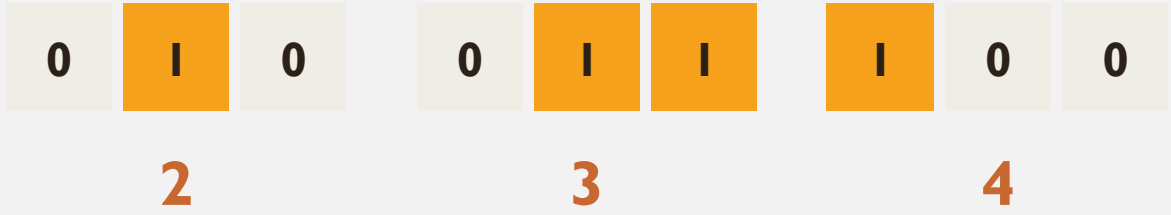
- **Two symbols, 0 and 1.** One binary digit is one bit; eight bits make one byte.
- **Place values are powers of 2:** … 128, 64, 32, 16, 8, 4, 2, 1.
- **Read value by adding the places that hold a 1,** as below.
- **Hardware uses it** because a circuit holds two states reliably: off or on, low or high voltage.



10011100 = 128 + 16 + 8 + 4 = 156 in decimal

Octal

- **Eight symbols, 0 to 7.** There is no digit 8 or 9.
- **Place values are powers of 8:** … 512, 64, 8, 1.
- **234 in octal =** (2×64) + (3×8) + (4×1) = 156 in decimal.
- **Three bits make one octal digit** ($2^3 = 8$), so 010 011 100 reads straight off as 234.
- **Used as shorthand for bit patterns,** such as Unix file permissions: 755 means rwxr-xr-x.



156 in binary, padded to nine bits and split in threes from the right. Each group is one octal digit.

Hexadecimal

- **Sixteen symbols, 0 to 9 then A to F,** where A = 10 and F = 15.
- **Place values are powers of 16:** … 4096, 256, 16, 1.
- **9C in hex =** (9×16) + (12×1) = 156 in decimal.
- **Four bits make one hex digit** (2^4 = 16), so a byte is always two digits: 1111 1111 = FF = 255.
- **Used where bytes matter:** memory addresses, colour codes, byte dumps.



The same byte split in fours. Each group is one hex digit, so 156 is 9C.

The four bases side by side

Every base works the same way: each digit is multiplied by its place value. Only the number of symbols changes.

	Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hex (base 16)
Symbols	0-9	0-1	0-7	0-9, A-F
How many symbols	10	2	8	16
Place values	1, 10, 100, 1000	1, 2, 4, 8	1, 8, 64, 512	1, 16, 256, 4096
The number 156	156	10011100	234	9C
Digits needed	3	8	3	2
Where you meet it	Everyday counting	Circuits and logic	File permissions	Addresses, colours

Converting between Bases

Why do we need to convert number bases?

- **Human readability:** Long strings of numbers are hard for us to read and keep track of. Hex and Octal are compact shorthand for binary.
- **Programming and Input/Output:** Programmers may need to convert base 10 data into other bases for low-level tasks like bit masks and memory management.
- **Specialised Applications:** Unix file permissions use octal to represent read, write and execute

Convert from base *b* to decimal

- Use general formula as we saw a moment ago…
- Convert d_i to decimal and multiply it by b^i, then sum the results:

$$(357)_8 = \quad + \quad +$$
$$= \quad \cdot + \quad +$$
$$= 192 + 40 + 7$$

Convert from base *b* to decimal

- Use general formula as we saw a moment ago…
- Convert d_i to decimal and multiply it by b^i, then sum the results:

$$
\begin{aligned}
(357)_8 &= 3 \cdot 8^2 + 5 \cdot 8^1 + 7 \cdot 8^0 \\
&= 3 \cdot 64 + 5 \cdot 8 + 7 \cdot 1 \\
&= 192 + 40 + 7 \\
&= (239)_{10}
\end{aligned}
$$

Convert from base *b* to decimal

- Use general formula as we did for positional notation:
- Convert d_i to decimal and multiply it by b^i, then sum the results:

$$(AE4)_{16} = A \cdot 16^2 + E \cdot 16^1 + 4 \cdot 16^0$$
$$= A \cdot 256 + E \cdot 16 + 4 \cdot 1$$
$$= 10 \cdot 256 + 14 \cdot 16 + 4 \cdot 1$$
$$= 2560 + 224 + 4$$
$$= (2788)_{10}$$

Note here that the hexadecimal "A" is equivalent to "10" in decimal

Convert from decimal to base b

- While number is not zero
  - Divide number by new base b (2, 8, 16…)
  - Note down remainder
  - Replace number by quotient

- The remainders listed in reverse order is the value in base b.

Calculator Time!

Convert from decimal to base *b*

example 1

Convert $(239)_{10}$ to base 8:

$$(239)_{10} / 8 = 29.875$$

Quotient = 29

Remainder = 0.875 * 8 (base) = 7

Convert from decimal to base *b*

example 1

Convert $(239)_{10}$ to base 8:

$(239)_{10}$ / 8 = 29 remainder **7**

29 / 8 = 3, remainder **5**

3 / 8 = 0, remainder **3**

$= (357)_8$

Convert from decimal to base *b*

example 2

Convert $(239)_{10}$ to base 16:

$(239)_{10}$ / 16 = 14 remainder **15 (F)**

14 / 16 = 0, remainder **14 (E)**

= $(EF)_{16}$

Convert from base *x* to base *y*

- Actually the same algorithm as with decimal:
  - While number is not zero
    - Divide number by new base b
    - Note down remainder
    - Replace number by quotient
  - Read remainders in reverse

- Note: Now we are doing division in base x.

Convert from base x to base y

example 3

Convert $(A6C)_{14}$ to base $(12)_{10}$ (actually base $(C)_{14}$):

$(A6C)_{14}$ / C = C3 remainder **4**

C3 / C = 10, remainder **3**

10 / C = 1, remainder **2**

1 / C = 0, remainder **1**

$= (1234)_{12}$

Why?

Note that these numbers are all base 14!

How's your long division in base 14?

Requires thinking in bases which aren't intuitive to us. Instead we usually just convert base x to decimal, and then convert from decimal to base y.

Convert from base x to base y (via decimal)

example 3 the same sum through decimal

Step 1: base 14 to decimal

$(A6C)_{14} = A\times14^2 + 6\times14^1 + C\times14^0$

$= 10\times196 + 6\times14 + 12\times1$

$= 1960 + 84 + 12$

$= 2056_{10}$

Multiply each digit by its place value, in decimal.

Step 2: decimal to base 12

2056 ÷ 12 = 171 remainder **4**

171 ÷ 12 = 14 remainder **3**

14 ÷ 12 = 1 remainder **2**

1 ÷ 12 = 0 remainder **1**

$= (1234)_{12}$

Read the remainders upwards.

Check: $(1234)_{12} = 1728 + 288 + 36 + 4 = 2056$. Same answer as dividing in base 14, but every calculation here was in decimal.

Cheats approach to convert between bases of power 2

- Octal:
  - Starting from the right, group digits into groups of 3.
  - Convert group by group to octal.

$(1001001101111)_2$				
1	001	001	101	111
1	1	1	5	7
$(11157)_8$				

- Hexadecimal:
  - Starting from the right, group digits into groups of 4.
  - Convert group by group to hexadecimal.

$(1001001101111)_2$			
1	0010	0110	1111
1	2	6	F
$(126F)_{16}$			

Goals for Today

- Categories of numbers and positional notation
- Overview of bases 2, 8, and 16
- Converting numbers between bases

CS-107 Lecture 3
Number Systems

Troy Astarte

Megan Venn-Wycherley